Reporte Técnico: Implementación de Estructuras de Datos

1. Diseño de las Estructuras de Datos

El sistema central (CatalogManager) fue diseñado bajo una arquitectura de índices múltiples, esto con la finalidad de hacer una comparación entre las estructuras de datos secuenciales y binarias. Un único objeto Product reside en la memoria dinámica (Heap), y es apuntado simultáneamente por cinco estructuras diferentes, optimizando las consultas según el atributo requerido.

- **Tabla Hash (Código de Barras):** Se implementó mediante arreglos dinámicos y resolución de colisiones por **encadenamiento** (Separate Chaining). Se eligió esta estructura principal porque el código de barras funge como la **Llave Primaria** (Primary Key) del sistema; es un identificador único que requiere acceso determinista e instantáneo en tiempo *O(1)* exacto para simulaciones de punto de venta, donde las búsquedas por rango no aplican. Se descartó explícitamente el uso de su alternativa matemática, el **Direccionamiento Abierto** (Open Addressing mediante Sondeo Lineal o Cuadrático), debido a dos limitantes arquitectónicas que impactan directamente en el rendimiento de un catálogo dinámico:
    - **La Complejidad de la Eliminación (El problema del Tombstone**): En un sistema donde los productos se eliminan constantemente (Opción 4 del menú), el Direccionamiento Abierto presenta un fallo estructural. Al eliminar un elemento, se rompe la cadena de saltos de las colisiones anteriores. Para solucionarlo, se requiere implementar marcas de borrado lógico (Tombstones o Lazy Deletion). A medida que se borran productos, la tabla se contamina de estas "lápidas", lo que degrada silenciosamente el tiempo de búsqueda de *O(1)* a *O(n)*, obligando al sistema a realizar un costoso rehashing para limpiar la memoria. Con el Encadenamiento, la eliminación es atómica y limpia: simplemente se desenlaza y destruye el nodo de la lista secundaria, sin afectar en absoluto al resto de los elementos de la ranura.
    - **Aglomeración Primaria (Primary Clustering):** A medida que la estructura se llena (Factor de Carga alto), el Direccionamiento Abierto agrupa los productos en bloques continuos de memoria. Esto provoca un efecto avalancha, donde cada nueva inserción tarda exponencialmente más en encontrar una ranura vacía. El Encadenamiento mitiga esto por completo al crecer verticalmente en la memoria dinámica (Heap), permitiendo que el sistema siga funcionando

de manera estable incluso si la tabla supera su capacidad óptima antes de ser redimensionada.

- **Árbol AVL (Nombres):** Se estructuró un árbol binario auto-balanceable. Es ideal para este atributo ya que permite búsquedas rápidas, mantiene los nombres estrictamente ordenados alfabéticamente para reportes (recorrido In-Order), y evita la degradación a listas degeneradas garantizando el balanceo mediante rotaciones simples y dobles.
- **Árbol B (Fechas de Expiración):** Se diseñó con un grado mínimo *t=3*. Se eligió para los rangos de fechas porque agrupa las claves en nodos multidimensionales. El grado *t=3* fue seleccionado deliberadamente para fines académicos, forzando escenarios de división (*split*) y fusión (*merge*) con volúmenes de datos pequeños, demostrando empíricamente la reestructuración de la base de datos.
- **Árbol B+ (Categorías):** Al igual que el Árbol B, opera con *t=3*, pero confina los datos exclusivamente en las hojas. Se seleccionó para las categorías porque permite búsquedas de rango masivas de forma iterativa *O(k)* mediante su lista enlazada a nivel de hojas, optimizando drásticamente la extracción de todos los productos de una misma categoría sin recorrer el árbol superior.
- **Listas Enlazadas (Secuencial y Ordenada):** Actúan como estructuras base de control de memoria y como contrapunto empírico para demostrar la ineficiencia de las estructuras lineales frente a las arbóreas.

2. Justificación Teórica de Complejidad (Big-O)

El diseño del sistema garantiza matemáticamente los siguientes tiempos de ejecución teóricos para las operaciones de búsqueda:

Estructura	Caso Promedio	Peor Caso	Justificación Matemática
Tabla Hash	$O(1)$	$O(n)$	El acceso mediante la función hash al índice del arreglo es instantáneo. El peor caso O(n) solo ocurre si todos los elementos colisionan en la misma ranura, lo cual se mitiga con una buena dispersión.

Árbol AVL	$O(\log n)$	$O(\log n)$	El factor de equilibrio (diferencia de alturas entre subárboles ≤1) asegura que la profundidad del árbol nunca exceda $1.44log_2 n$. Cada paso descarta la mitad del espectro de búsqueda.
Árbol B / B+	$(O(log_t n)$	$(O(log_t n)$	El crecimiento "ancho" del árbol reduce la altura drásticamente. Buscar requiere descender por niveles acotados logarítmicamente según el grado *t*, complementado con una búsqueda interna en el nodo de tamaño O(t).
Lista Enlazada	$O(n)$	$O(n)$	Al carecer de índices o saltos de memoria, requiere inspeccionar iterativamente cada nodo n hasta encontrar el objetivo o agotar la estructura.

3. Metodología de Medición (Benchmarking)

Para asegurar la validez científica de la comparación entre el desempeño teórico y el empírico, se implementó un motor interno de *benchmarking* utilizando la librería <chrono> de C++ con precisión de microsegundos (high_resolution_clock).

La metodología consistió en:

1. **Tamaño de Muestra (N):** Se define un número N de consultas a realizar por ciclo.
2. **Repeticiones (M):** Para absorber las anomalías del procesador y del sistema operativo (ruido del planificador de tareas), cada experimento de N consultas se repite M veces, calculando la media aritmética estricta.
3. **Casos de Prueba Aislados:**
    - *Búsqueda Exitosa Aleatoria:* Arreglo mezclado (Fisher-Yates / std::shuffle) para simular acceso arbitrario.
    - *Búsqueda Fallida:* Elementos inexistentes ("ZZZ_PRODUCTO_999") para forzar a los algoritmos a recorrer su peor caso.

- *Extremos (Mejor/Peor Caso Estructural):* Búsquedas dirigidas específicamente a la primera y última posición alfabética.

4. Resultados

Tabla de Tiempos Inserción y Eliminación (en microsegundos, µs)

(Parámetros de prueba: N=1000. Tiempos promediados por consulta)

Escenario de Prueba	Tabla Hash	Lista No Ordenada	Lista Odenada	Arbol AVL	Arbol B	Arbol B+
1. Inserción	0.42 µs	0.10 µs	7.45 µs	1.04 µs	1.69 µs	0.29 µs
2. Eliminación	2.1 µs	1E+02 µs	60 µs	5.8 µs	2 µs	10 µs

Análisis de Resultados: Costo de Inserción vs. Costo de Eliminación

Las métricas obtenidas revelan el principio fundamental del diseño de las estructuras de datos: el *trade-off* (compromiso) estructural. Indexar la información tiene un costo computacional inicial, pero es una inversión necesaria para garantizar la escalabilidad a largo plazo.

1. Fase de Inserción: El Costo de Indexación (Promedio por producto)

Al promediar el esfuerzo de insertar 1,000 productos, observamos cómo el mantenimiento del orden impacta a la CPU:

- El Espejismo Lineal ($O(1)$): La Lista Secuencial (0.10 µs) es la más rápida. Al insertar siempre en la cabeza (insertFront), no hace comparaciones, solo asigna memoria. Es ideal para *logs* rápidos, pero inútil como índice.
- El Colapso del Orden Lineal ($O(n)$): La Lista Ordenada (7.45 µs) demuestra ser ineficiente para cargas masivas. En promedio, tardó 74 veces más que la secuencial. Esto ocurre porque cada nuevo producto obliga a iterar sobre los anteriores para encontrar su posición alfabética, degradando el proceso a un costo acumulado cuadrático $O(n^2)$.

- La Inversión Estructural ($O(\log n)$).El Árbol B (1.69 µs) y el Árbol AVL (1.04 µs) representan el costo real de mantener un índice robusto. Dedican microsegundos valiosos a dividir nodos (*splits*) y ejecutar rotaciones de balanceo.
- El Fenómeno de Baja Cardinalidad: El Árbol B+ (0.29 µs) fue sorprendentemente rápido. Esto se justifica teóricamente porque, aunque hay 1,000 productos, existen muy pocas categorías únicas. El árbol no crece verticalmente; simplemente encuentra la hoja existente en tiempo mínimo y añade el producto a la lista enlazada inferior en $O(1)$.

2. Fase de Eliminación: El Retorno de Inversión

Al eliminar un producto específico, los paradigmas de rendimiento se invierten por completo, exponiendo el verdadero valor de los árboles y tablas hash frente a las listas lineales:

- La Caída de las Listas ($O(n)$): La Lista Secuencial, que presumía 0.10 µs en inserción, colapsó a 100 µs (1e+02 µs en notación del compilador) al eliminar. Al no tener un índice, el algoritmo operó a ciegas, viéndose obligado a recorrer cientos de nodos secuencialmente hasta encontrar el objetivo. La Lista Ordenada sufrió el mismo destino (60 µs).
- Supremacía del Acceso Directo ($O(1)$): La Tabla Hash (2.1 µs) confirmó su idoneidad como llave primaria. Mediante la función matemática de dispersión, localizó el código de barras y eliminó el nodo casi instantáneamente, sin necesidad de navegar el catálogo.
- Mantenimiento Arbóreo Eficiente ($O(\log n)$): Los árboles demostraron que su costo de inserción valió la pena. El Árbol AVL (5.8 µs) y el Árbol B (2 µs) resolvieron la búsqueda y eliminación en un solo dígito de microsegundos. El Árbol B+ (10 µs) tomó ligeramente más tiempo debido a que la eliminación requirió desvincular el producto de la lista enlazada a nivel de hoja, pero manteniéndose muy por debajo de las estructuras lineales.

```
--- Resumen de Carga y Tiempos de Insercion ---
Insertados con exito: 1000
Omitidos por duplicado: 0

[Tiempos PROMEDIO de Insercion por producto]
1. Tabla Hash        (O(1))     : 0.42 us
2. Lista Secuencial  (O(1))     : 0.10 us
3. Lista Ordenada    (O(n))     : 7.45 us
4. Arbol AVL         (O(log n)): 1.04 us
5. Arbol B  (Fechas) (O(log n)): 1.69 us
6. Arbol B+ (Categ.) (O(log n)): 0.29 us
-----------------------------------------------
```

```
--- Producto Eliminado Exitosamente ---
Tiempos de re-estructuracion:
 - Hash (O(1))     : 2.1 us
 - Lista (O(n))    : 1e+02 us
 - Lista Ordenada : 60 us
 - AVL (O(log n)) : 5.8 us
 - BTree (O(n))    : 2 us
 - BPlusTree (O(log n)) : 10 us

Producto eliminado de TODAS las estructuras correctamente.
```

Tabla de Tiempos Búsqueda (en microsegundos, µs)

(Parámetros de prueba: N=20, M=5. Tiempos promediados por consulta)

Escenario de Prueba	Lista No Ordenada	Lista Ordenada	Árbol AVL
1. Búsqueda Exitosa (Aleatoria)	1.71 µs	4.25 µs	0.41 µs
2. Búsqueda Fallida (No existe)	11.72 µs	10.57 µs	0.47 µs
3. Extremo Inferior (Primero)	0.38 µs	0.16 µs	0.58 µs
4. Extremo Superior (Último)	0.16 µs	12.48 µs	0.74 µs

Análisis de Resultados:

Los datos obtenidos confirman empíricamente la teoría.

El Peor Caso Absoluto (Búsqueda Fallida)

El **Caso 2** es la prueba de fuego de cualquier estructura de datos. Cuando buscas algo que no existe, obligas a la estructura a llegar a su límite.

- Listas (11.72 µs y 10.57 µs): La Lista No Ordenada sufre su peor escenario O(n), viéndose forzada a iterar sobre cada uno de los elementos para asegurar que el producto no está. La Lista Ordenada mejora ligeramente porque puede aplicar un "aborto temprano" (frenar cuando el nombre actual es alfabéticamente mayor al buscado), pero sigue atrapada en un tiempo lineal altísimo.
- Árbol AVL (0.47 µs): Demuestra su abrumadora superioridad. Al buscar algo que no existe, el árbol simplemente desciende logarítmicamente hasta chocar con un puntero nulo (nullptr). Descarta la mitad del catálogo en cada paso, resolviendo la consulta 25 veces más rápido que la lista no ordenada.

La Consistencia del Árbol (La magia del Balanceo)

Si se observa la columna del Árbol AVL de arriba a abajo (0.41 µs, 0.47 µs, 0.58 µs, 0.74 µs), notará que los tiempos son casi idénticos en todos los escenarios.

- Esto demuestra empíricamente que las rotaciones del AVL funcionan a la perfección. La profundidad del árbol está matemáticamente acotada a $O(\log n)$. No importa si buscas el primer elemento, el último, uno en medio o uno que no existe; el motor garantiza un tiempo de respuesta predecible y estable.

El Espejismo del Mejor Caso (Extremo Inferior)

En el **Caso 3**, vemos un fenómeno muy interesante: la Lista Ordenada marcó apenas 0.16 µs, superando al Árbol AVL.

- Esto no significa que la lista sea mejor, sino que tuvimos un caso O(1) por casualidad estructural. El "Extremo Inferior" (el primer elemento alfabético) vive exactamente en el puntero head de la Lista Ordenada. La computadora lo encuentra en el primer intento. El AVL, por su parte, tuvo que gastar 0.58 µs bajando por todos los punteros de su rama izquierda hasta encontrar la hoja más pequeña.

El Colapso en el Extremo Superior

El **Caso 4** expone el gran talón de Aquiles de las Listas Ordenadas.

- Lista Ordenada (12.48 µs): Al buscar el elemento alfabéticamente más grande (ej. "Yogurt"), la lista sufre un castigo masivo. Como está ordenado de la 'A' a la 'Z', el último elemento es la cola (Tail). El algoritmo se ve obligado a atravesar todos los punteros desde el head hasta el final, marcando el tiempo más lento de todo el experimento.
- El fenómeno de la Lista No Ordenada (0.16 µs): Porque la función insertFront apila los datos como una pila (LIFO). El último producto que se cargó del archivo CSV quedó en la cabeza (head) de la lista no ordenada. Al buscarlo, el sistema lo encontró al instante en O(1).

```
==================================================================
 INICIANDO BENCHMARK: N = 20 consultas | M = 5 repeticiones
==================================================================
--- 1. CASO: BUSQUEDA EXITOSA ALEATORIA ---
Lista No Ordenada: 1.71 us
Lista Ordenada   : 4.251 us
Arbol AVL        : 0.412 us

--- 2. CASO: BUSQUEDA FALLIDA ---
Lista No Ordenada: 11.723 us (Recorre TODO buscando)
Lista Ordenada   : 10.575 us (Frena por orden alfabetico)
Arbol AVL        : 0.477 us (Baja rapido a una hoja vacia)

--- 3. CASO: EXTREMO INFERIOR (Primero: Aceite Capullo) ---
Lista No Ordenada: 0.38 us
Lista Ordenada   : 0.16 us
Arbol AVL        : 0.58 us

--- 4. CASO: EXTREMO SUPERIOR (Ultimo: Yogurt Yoplait) ---
Lista No Ordenada: 0.16 us
Lista Ordenada   : 12.48 us
Arbol AVL        : 0.74 us
==================================================================
```

Conclusión

El análisis demuestra de forma concluyente que las estructuras no lineales (Árboles y Hash) son estrictamente necesarias para el desarrollo de software empresarial. Aunque exigen un costo computacional y de memoria superior durante su

construcción, previenen el colapso del sistema (cuellos de botella por $O(n)$) durante las operaciones críticas del ciclo de vida del dato: la búsqueda y la eliminación.

El análisis del benchmarking confirma empíricamente la superioridad de las estructuras no lineales en estructuras de datos. Mientras que la Lista Ordenada mostró picos de rendimiento inconsistentes (fluctuando desde 0.16 µs en el mejor caso hasta 12.48 µs en el peor caso estructural), el Árbol AVL mantuvo una latencia estrictamente acotada por debajo de 0.80 µs en todos los escenarios. Esta consistencia valida que el costo de procesamiento adicional al realizar rotaciones de balanceo en tiempo de inserción se amortiza ampliamente en tiempo de consulta, garantizando resoluciones de latencia ultra-baja en O(log n) sin importar el orden alfabético ni la existencia del dato consultado.